CENTRO UNIVERSITÁRIO UNIFECAF

Análise e Desenvolvimento de Sistemas

Diego Silva Araujo

Relatório Teórico:

Monitoramento e Observabilidade da Plataforma E-Fecaf Global: Prometheus, Grafana, Alertas e Testes de Carga para a Black Friday

Taboão da Serra, SP
2026

Diego Silva Araujo

Relatório Teórico:

Monitoramento e Observabilidade da Plataforma E-Fecaf Global: Prometheus, Grafana, Alertas e Testes de Carga para a Black Friday

Trabalho apresentado como requisito de avaliação da disciplina Quality Assurance do Curso de Graduação em Análise e Desenvolvimento de Sistemas do Centro Universitário UniFECAF.

Tutor(a): Felipe Bonatto

Taboão da Serra, SP
2026

SUMÁRIO

1. Introdução 4
2. Contextualização do Problema e Proposta 4
3. Arquitetura e Tecnologias 4
4. Passos Realizados 5
5. Métricas Monitoradas 7
6. Testes de Carga e Estresse 7
7. Visualizações 9
8. Análise dos Resultados e Otimização 15
9. Justificativa das Decisões Técnicas 16
10. Conclusão 17
Referências 17

1. Introdução

Este relatório documenta a implantação de uma solução de monitoramento e observabilidade para a E-Fecaf Global, uma plataforma de comércio eletrônico, realizada na disciplina Quality Assurance. O objetivo foi tornar visível o comportamento da plataforma durante um pico de acessos semelhante ao de uma Black Friday, medindo desempenho, disponibilidade e saturação dos recursos, e emitindo alertas automáticos quando limites críticos são ultrapassados.

Para isso, foi montado um laboratório completo em contêineres Docker: uma loja virtual em Spring Boot com banco PostgreSQL, coleta de métricas com Prometheus, verificação externa de disponibilidade com Blackbox Exporter, painéis no Grafana, envio de alertas ao Telegram pelo Alertmanager e simulação de clientes com o Locust.

O documento descreve o problema, a arquitetura e as tecnologias, os passos executados, as métricas monitoradas, os testes de carga e estresse, a análise dos resultados com as otimizações propostas e testadas, a justificativa das decisões técnicas e a conclusão.

2. Contextualização do Problema e Proposta

Durante a última Black Friday, a E-Fecaf Global sofreu lentidão e quedas, mas a equipe não conseguiu identificar a causa: não havia métricas, histórico nem alertas. Sem monitoramento, não era possível responder a perguntas básicas, como quais páginas ficaram lentas, se o problema estava na aplicação ou no banco de dados, a partir de quantos clientes simultâneos a plataforma começou a falhar e quando a equipe deveria ter sido avisada.

A proposta foi implantar uma pilha de observabilidade baseada nos quatro sinais de ouro do monitoramento (latência, tráfego, erros e saturação) e validá-la com testes que reproduzem o pico de acessos. Para que o estudo fosse reproduzível, o laboratório recria de forma controlada três gargalos típicos de lojas virtuais: um pool pequeno de conexões com o banco (5 conexões), um checkout que mantém a conexão aberta enquanto aguarda o gateway de pagamento (latência simulada de 300 ms) e uma busca de produtos sem índice sobre um catálogo de 50 mil itens.

3. Arquitetura e Tecnologias

O fluxo de dados segue a sequência abaixo:

```
Locust (clientes) -> Loja Spring Boot -> PostgreSQL
Loja (/actuator/prometheus) + Blackbox (health check) -> Prometheus -> Grafana
Prometheus (regras de alerta) -> Alertmanager -> Telegram
```

Tecnologia	Função no projeto
Spring Boot 3 e Java 21	Loja virtual com vitrine, detalhe de produto, busca, carrinho e checkout
Spring Boot Actuator e Micrometer	Exposição de métricas da aplicação (requisições, tempos, status HTTP, JVM, CPU e pool de conexões) no formato do Prometheus
PostgreSQL 16	Banco de dados com 50 mil produtos gerados automaticamente e tabela de pedidos
Prometheus 2.54	Coleta das métricas a cada 5 segundos, armazenamento do histórico e avaliação das regras de alerta
Blackbox Exporter	Teste externo de disponibilidade, como um cliente faria (o site responde?)

Tecnologia	Função no projeto
Alertmanager	Agrupamento dos alertas e envio de notificações ao Telegram, incluindo a mensagem de resolução
Grafana 11	Dashboard com resumo executivo e painéis técnicos, provisionado automaticamente
Locust 2.31	Simulação de clientes e do perfil de carga da Black Friday
Docker e Docker Compose	Execução de todos os serviços em contêineres, com um único comando
Git e GitHub	Versionamento do projeto e publicação do repositório

Todo o laboratório foi executado em um servidor Ubuntu na rede local, acessado por SSH. As ferramentas web (loja, Prometheus, Alertmanager, Grafana e Locust) são acessadas pelo navegador do computador pessoal, usando o endereço IP do servidor.

4. Passos Realizados

4.1 Repositório Git e GitHub

O projeto foi versionado com Git e publicado no GitHub. O arquivo .gitignore impede que o token do bot do Telegram seja enviado ao repositório, e o arquivo .gitattributes padroniza as quebras de linha no formato Linux, usado pelo servidor.

```
git init -b main
git add -A
git commit -m "Laboratório de monitoramento da E-Fecaf Global"
git remote add origin https://github.com/Dieggo775/efecaf-monitoramento.git
git push -u origin main
```

4.2 Aplicação instrumentada

A loja foi desenvolvida em Spring Boot com os endpoints /api/produtos, /api/produtos/{id}, /api/busca, /api/carrinho e /api/checkout. A dependência micrometer-registry-prometheus, junto com o Actuator, publica as métricas em /actuator/prometheus. O histograma de tempos de resposta foi habilitado para permitir o cálculo de percentis (p50, p95 e p99) no Prometheus. O trecho abaixo mostra as configurações principais:

```
spring.datasource.hikari:
  maximum-pool-size: ${DB_POOL_SIZE:5}      # gargalo proposital
  connection-timeout: 2000                  # espera máxima por conexão (ms)
management:
  endpoints.web.exposure.include: health,info,prometheus,metrics
  metrics.distribution.percentiles-histogram.http.server.requests: true
```

4.3 Orquestração com Docker Compose

Os sete serviços (postgres, app, prometheus, blackbox, alertmanager, grafana e locust) foram definidos em um único arquivo docker-compose.yml. A aplicação é compilada dentro de um contêiner Maven, de modo que não é necessário instalar Java no servidor. A loja recebeu limite de 1 CPU e 768 MB de memória, para reproduzir um servidor de capacidade limitada. A política restart: unless-stopped faz os contêineres voltarem sozinhos após uma queda de energia.

```
docker compose up -d --build
docker compose ps
```

4.4 Coleta de métricas com Prometheus e Blackbox

O Prometheus coleta as métricas da loja e do Blackbox a cada 5 segundos. O Blackbox acessa o endpoint de liveness da loja pelo lado de fora, como um cliente, e informa se o site respondeu (probe_success).

```
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: efecaf-app
    metrics_path: /actuator/prometheus
    static_configs: [{ targets: ["app:8080"] }]
  - job_name: blackbox
    metrics_path: /probe
    params: { module: [http_2xx] }
```

4.5 Dashboard no Grafana

A fonte de dados e o dashboard E-Fecaf Global – Saúde da Plataforma são criados automaticamente na inicialização do Grafana (provisionamento). O dashboard tem 12 painéis:

- **Resumo executivo:** disponibilidade nos últimos 5 minutos, requisições por segundo, tempo de resposta p95 e taxa de erros, com cores verde, amarela e vermelha conforme os limites;
- **Tráfego por endpoint e status HTTP:** mostram quais partes da loja recebem mais acessos e como se distribuem as respostas 2xx, 4xx e 5xx;
- **Tempo de resposta (p50, p95 e p99) e p95 por endpoint:** indicam a experiência do cliente e apontam o endpoint mais lento;
- **Taxa de erros (5xx):** com a linha do limite de alerta de 2%;
- **Pool de conexões do banco:** conexões em uso, livres, máximo e requisições esperando conexão, principal indicador de saturação do banco;
- **CPU e memória (heap da JVM):** saturação do servidor da aplicação.

4.6 Alertas e notificações no Telegram

As regras de alerta ficam no arquivo prometheus/alertas.yml. O campo for define por quanto tempo a condição precisa durar antes de disparar, o que evita alarmes falsos causados por picos de poucos segundos. Os tempos foram reduzidos para a demonstração; os valores recomendados para produção estão comentados no arquivo.

Alerta	Dispara quando	Severidade	for (lab / produção)
SiteForaDoAr	O health check externo falha	Crítico	15 s / 1 min
LatenciaAlta	p95 acima de 1 s	Aviso	30 s / 5 min
LatenciaCritica	p95 acima de 2 s	Crítico	30 s / 2 min
TaxaDeErrosAlta	Erros 5xx acima de 2% das requisições	Crítico	30 s / 5 min
PoolDoBancoSaturado	90% ou mais das conexões do banco em uso	Crítico	30 s / 5 min

Alerta	Dispara quando	Severidade	for (lab / produção)
CPUAlta	CPU da aplicação acima de 80%	Aviso	1 min / 10 min
MemoriaAlta	Heap da JVM acima de 85%	Aviso	1 min / 10 min

O Alertmanager agrupa os alertas por nome, aguarda 10 segundos para juntar alertas parecidos em uma única mensagem e envia ao Telegram tanto o disparo (🚨) quanto a resolução (✅). O token do bot fica em um arquivo separado (alertmanager/secrets/telegram_token), lido pelo parâmetro bot_token_file e excluído do Git.

5. Métricas Monitoradas

As métricas foram organizadas segundo os quatro sinais de ouro. A tabela mostra cada sinal, a métrica usada e a consulta PromQL correspondente:

Sinal	O que mede	Consulta PromQL (resumida)
Latência	Tempo de resposta p95	histogram_quantile(0.95, sum by (le) (rate(http_server_requests_seconds_bucket[1m])))
Tráfego	Requisições por segundo	sum(rate(http_server_requests_seconds_count[1m]))
Erros	Percentual de respostas 5xx	rate(...count{status=~"5.."}[1m]) / rate(...count[1m])
Saturação	Conexões do banco em uso	hikaricp_connections_active / hikaricp_connections_max
Saturação	Requisições esperando conexão	hikaricp_connections_pending
Disponibilidade	Site responde a um cliente externo	probe_success{job="blackbox"}

As requisições do próprio monitoramento (/actuator) são excluídas das consultas, para que os números reflitam apenas o tráfego dos clientes.

6. Testes de Carga e Estresse

6.1 Metodologia

Os testes foram realizados em 13/11/2026 com o Locust, que simula clientes navegando na loja. Cada cliente espera de 1 a 3 segundos entre as ações e escolhe as operações com os seguintes pesos: vitrine (5), detalhe de produto (4), busca (3), carrinho (2) e checkout (1). O perfil PicoBlackFriday controla a quantidade de clientes ao longo de 5 minutos:

Fase	Tempo	Usuários simultâneos
Manhã tranquila	0:00 – 1:00	50
Início da promoção	1:00 – 2:00	200
Pico (meia-noite da Black Friday)	2:00 – 3:30	800
Pós-pico	3:30 – 5:00	100

Os números de cada teste foram obtidos de duas fontes: o Locust, que mede a experiência do cliente, e o Prometheus, que mede o comportamento no servidor durante a fase de pico. Os relatórios completos do Locust e os arquivos CSV estão na pasta resultados/ do repositório.

6.2 Teste 1: carga normal (linha de base)

Com 50 usuários simultâneos durante 3 minutos, a loja permaneceu saudável: **0% de falhas** em 4.299 requisições, p95 de 0,34 s e cerca de 24 requisições por segundo. O pool de conexões chegou a 4 das 5 conexões, sem fila, e nenhum alerta disparou. Os endpoints mais lentos já eram o checkout (0,37 s, por causa do pagamento) e a busca (0,28 s).

6.3 Teste 2: pico da Black Friday

No perfil PicoBlackFriday, a plataforma entrou em colapso ao chegar a 800 usuários. Ao longo do teste, **31,2% das requisições falharam**, e no pico a taxa de erros no servidor chegou a 54,0%. O p95 percebido pelo cliente foi de 4,8 s. O painel do pool mostrou as 5 conexões ocupadas e até 194 requisições esperando uma conexão livre; as que esperaram mais de 2 segundos receberam erro 500. Quatro alertas dispararam automaticamente: LatenciaAlta, LatenciaCritica, TaxaDeErrosAlta e PoolDoBancoSaturado.

6.4 Teste 3: estresse (limite da plataforma)

Neste teste, a carga subiu continuamente até 1.500 usuários, à razão de 20 novos usuários por segundo. A tabela mostra a evolução nos primeiros 80 segundos:

Usuários	Req/s	Falhas	p95 (cliente)
220	45,2	0,0%	0,7 s
420	72,9	6,2%	2,2 s
620	119,6	44,6%	2,7 s
820	124,9	54,8%	4,3 s
1.020	126,6	54,3%	5,6 s
1.220	127,5	53,2%	6,8 s
1.420	126,1	52,2%	8,4 s
1.500	122,0	55,5%	10,0 s

O limite atual da plataforma ficou **entre 220 e 420 usuários simultâneos**: com 420 usuários, a taxa de falhas passou de 5% (6,2%) e o p95 chegou a 2,2 s. A partir daí, a vazão estabilizou em cerca de 130 requisições por segundo, independentemente do número de usuários, sinal claro de saturação. Com 1.500 usuários, até o health check externo deixou de responder a tempo, e o alerta SiteForaDoAr disparou: para o cliente, o site estava fora do ar. Nessa fase, a aplicação também deixou de responder à coleta do Prometheus, o que aparece como uma lacuna nos gráficos (Figura 5).

6.5 Teste de queda e notificação no Telegram

Para validar o alerta de disponibilidade, o contêiner da loja foi parado de propósito. O alerta SiteForaDoAr passou para o estado Pending em cerca de 10 segundos e para Firing após 30 segundos, e a mensagem 🚨 chegou ao Telegram. Com a loja religada, o alerta foi resolvido em cerca de 10 segundos e chegou a mensagem ✅ RESOLVIDO. O contador do Alertmanager registrou 2 notificações enviadas e nenhuma falha.

7. Visualizações

As figuras a seguir mostram o monitoramento em funcionamento durante os testes.

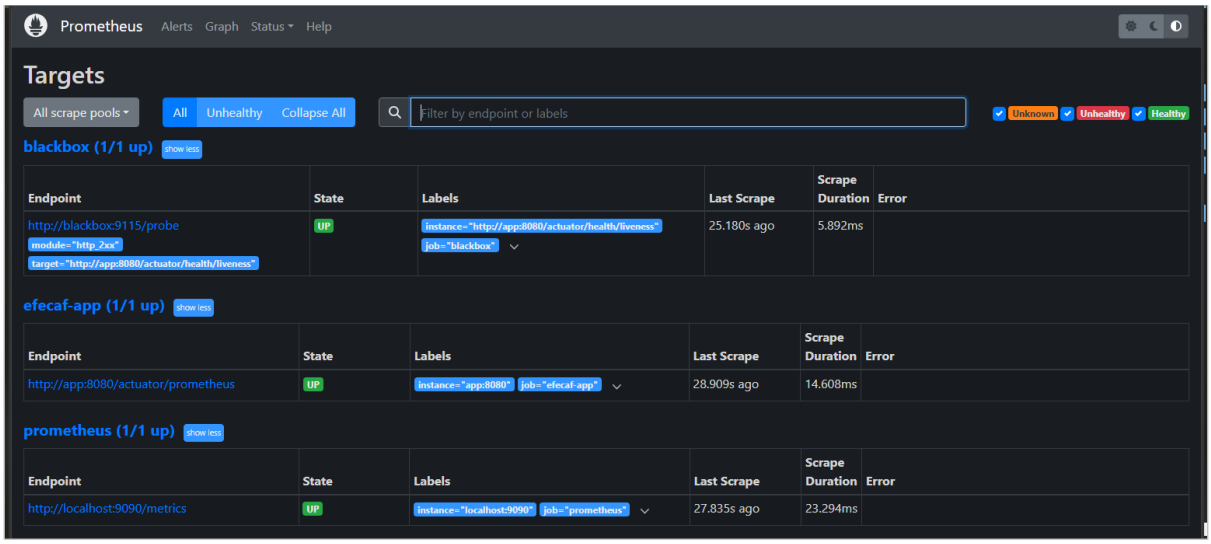


Figura 1 – Alvos monitorados pelo Prometheus: aplicação, Blackbox e o próprio Prometheus, todos UP.



Figura 2 – Dashboard E-Fecaf Global – Saúde da Plataforma com carga normal (Teste 1, 50 usuários).

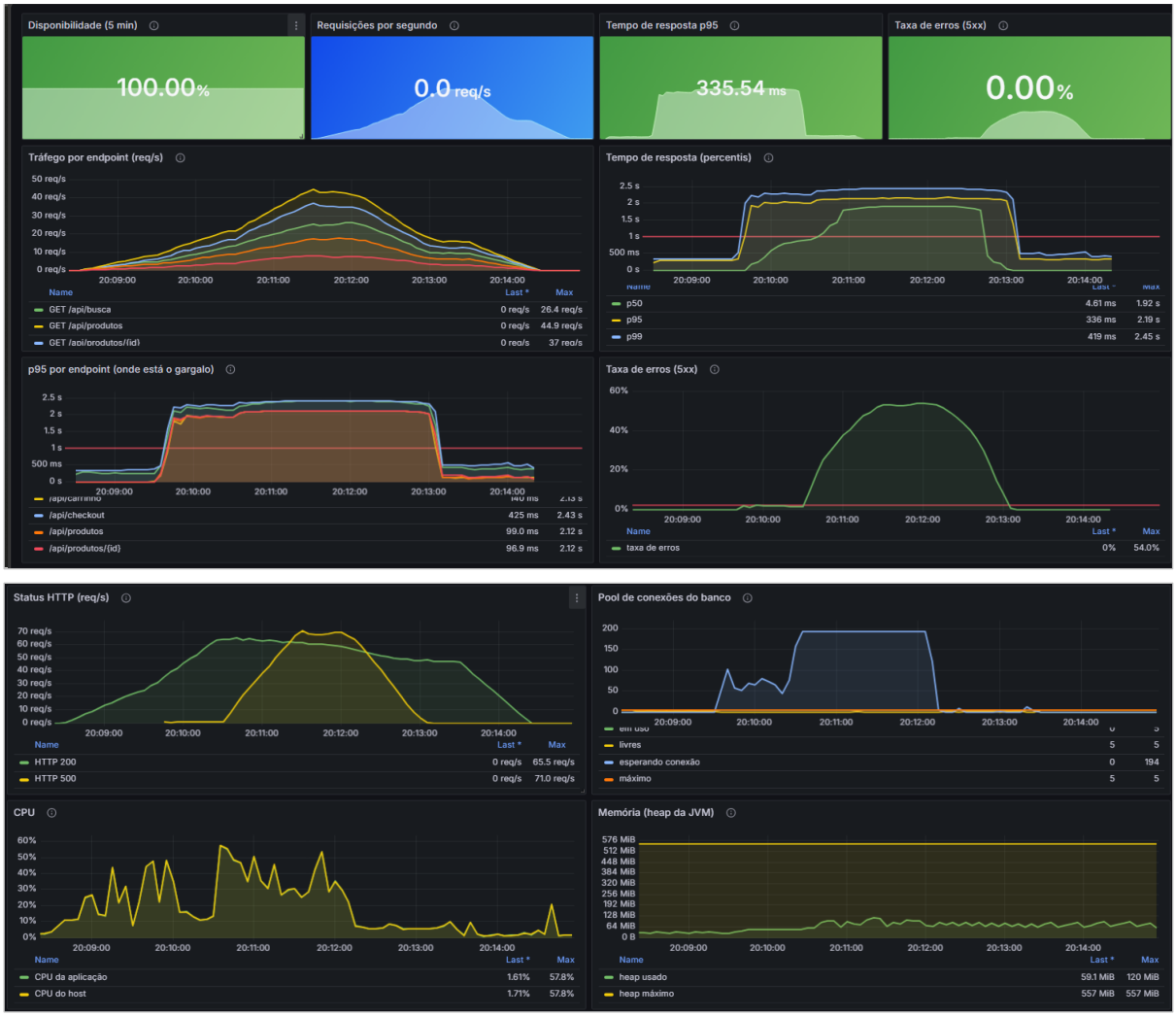


Figura 3 – Dashboard no pico da Black Friday (Teste 2): até 194 requisições esperando conexão, p95 acima de 2 s e taxa de erros de 54%.

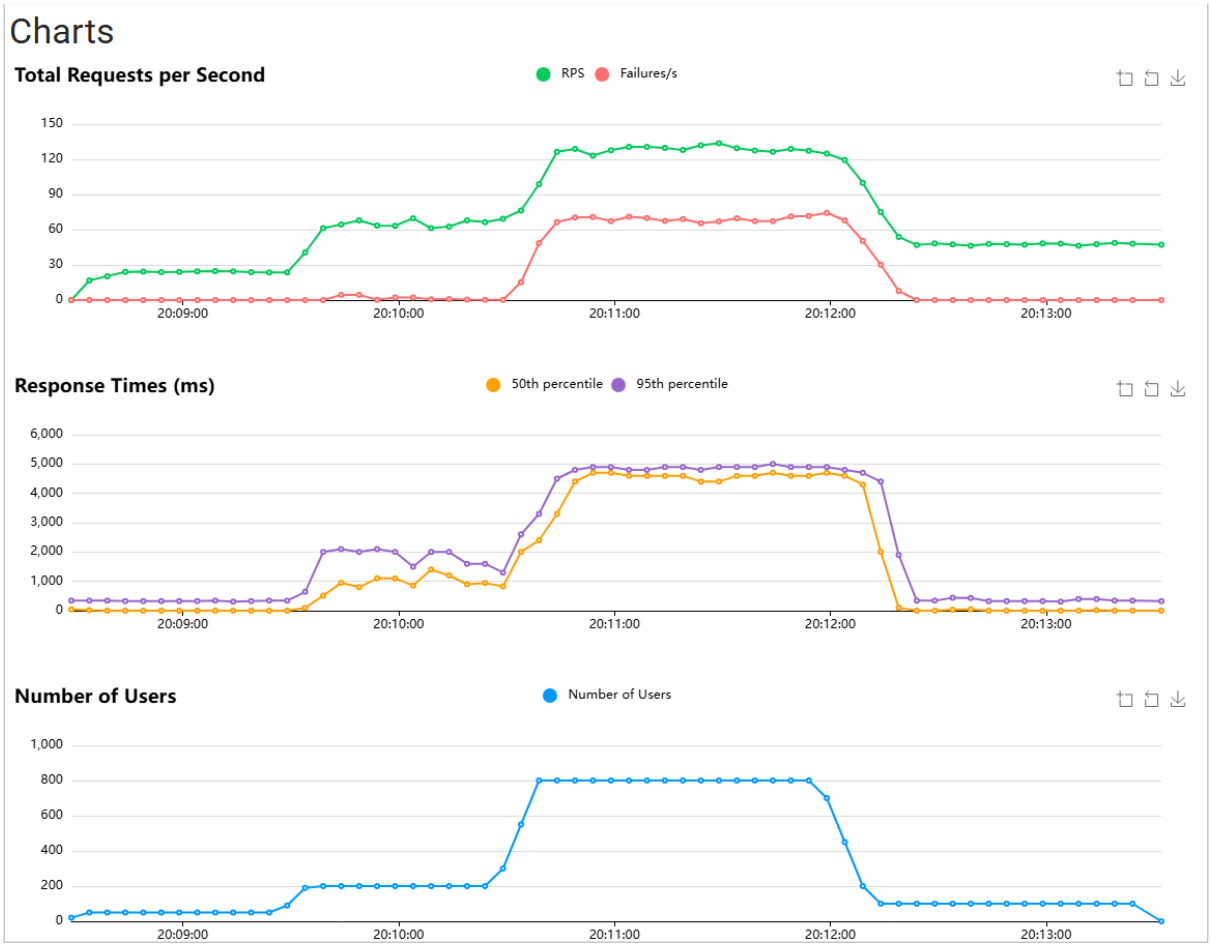


Figura 4 – Relatório do Locust no Teste 2: requisições e falhas por segundo, tempos de resposta (p50 e p95) e número de usuários.



Figura 5 – Teste de estresse (Teste 3): erros acima de 50% e p95 de 2,5 s. A lacuna entre 20:17 e 20:20 ocorre porque, com 1.500 usuários, a própria aplicação deixou de responder à coleta de métricas.

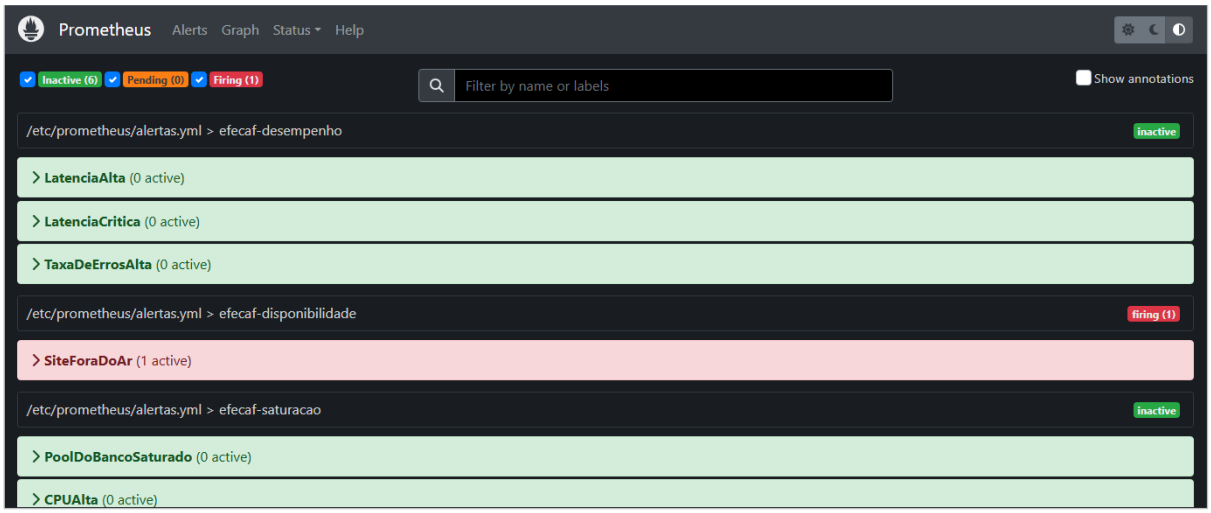


Figura 6 – Alerta SiteForaDoAr disparado (Firing) durante o teste de queda.

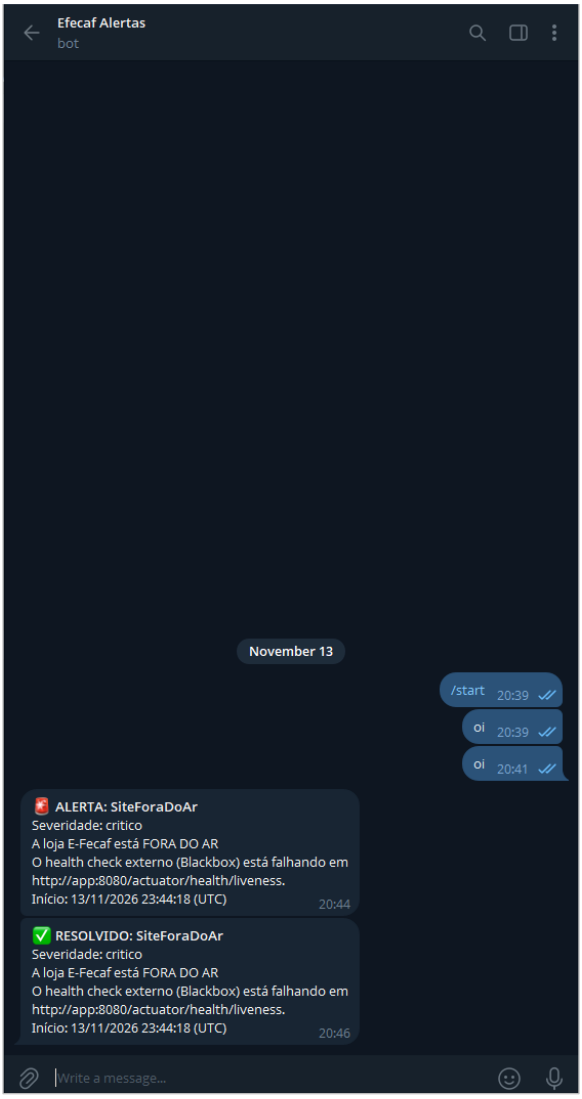


Figura 7 – Notificações de disparo (🚨) e de resolução (✅) recebidas no Telegram.

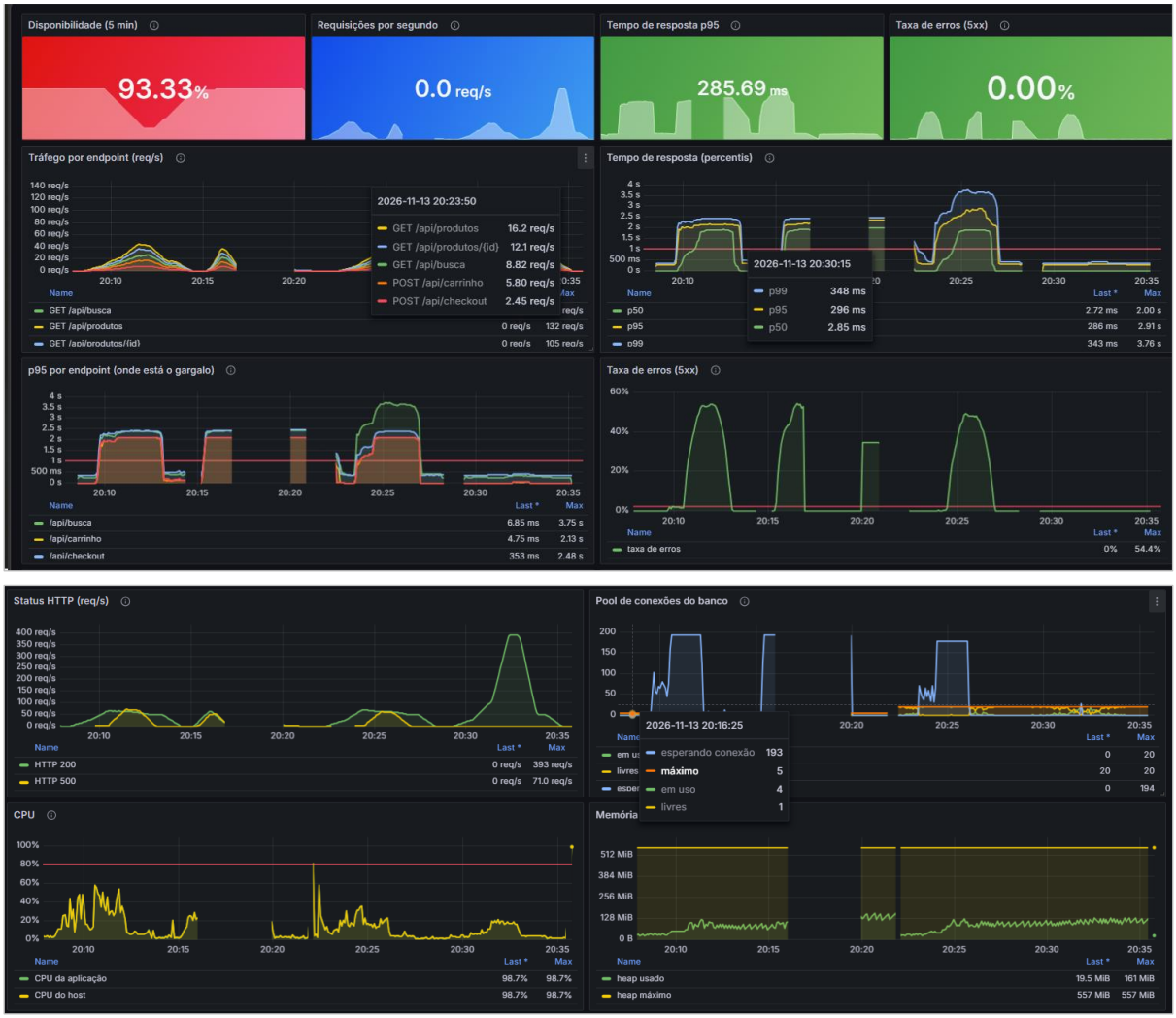


Figura 8 – Da esquerda para a direita: Teste 2 (antes), Teste 3 (estresse), Etapa 8.1 (pool 20 e 2 CPUs) e Etapa 8.2 (índices). Apenas o último pico não tem erros nem fila no pool.

8. Análise dos Resultados e Otimização

8.1 Primeiro palpite: mais conexões e mais CPU

Como o painel do pool mostrava as conexões esgotadas, a primeira hipótese foi aumentar o pool de 5 para 20 conexões e a CPU da aplicação de 1 para 2 núcleos. O resultado quase não mudou: as falhas caíram apenas de 31,2% para 27,1%, e o pool continuou saturado, agora com 20 conexões em uso e até 179 requisições na fila.

O monitoramento mostrou o motivo. O painel de p95 por endpoint apontava a busca como a operação mais lenta, e a CPU do contêiner do PostgreSQL chegou a 323% (mais de três núcleos inteiros). Com mais conexões, o banco apenas passou a executar mais buscas pesadas ao mesmo tempo. A análise da consulta mostrou que cada busca lia os 50 mil produtos (full scan) e consumia cerca de 215 ms de CPU.

8.2 Correção da causa: índices na busca

Foram criados três índices: dois do tipo GIN com trigramas (extensão pg_trgm), que permitem buscas com LIKE '%termo%' sem ler a tabela inteira, e um índice por preço, que permite ao banco encontrar

os 20 primeiros resultados já ordenados. O tempo de cada busca no banco caiu de cerca de 215 ms para cerca de 2 ms.

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_produto_nome_trgm ON produto
  USING gin (lower(nome) gin_trgm_ops);
CREATE INDEX idx_produto_descricao_trgm ON produto
  USING gin (lower(descricao) gin_trgm_ops);
CREATE INDEX idx_produto_preco ON produto (preco);
```

A tabela compara os três testes de pico (perfil PicoBlackFriday):

Métrica	Antes (pool 5, 1 CPU)	8.1 (pool 20, 2 CPUs)	8.2 (+ índices)
Taxa de falhas (Locust)	31,2%	27,1%	**0,0%**
p95 percebido pelo cliente	4,8 s	5,2 s	**0,31 s**
Req/s máximas atendidas	133	124	**393**
Requisições atendidas	21.517	20.948	47.869
Fila do pool (máx.)	194	179	29
CPU do PostgreSQL no pico	189%	323%	175%
Alertas disparados	4	4	**nenhum**

Com os índices, a plataforma atravessou o pico de 800 usuários **sem nenhuma falha**, com p95 de 0,31 s e capacidade 2,9 vezes maior. Nenhum alerta disparou. O resultado mostra o principal valor do monitoramento: sem os painéis, a equipe teria investido em mais servidores e mais conexões, que não resolveram o problema.

8.3 Outras propostas de otimização

- **Tirar o pagamento de dentro da transação:** o checkout não deve segurar a conexão do banco enquanto aguarda o gateway; a cobrança pode ser feita antes ou de forma assíncrona;
- **Fila para o checkout:** pedidos entram em uma fila (por exemplo, RabbitMQ ou Kafka) e são processados no ritmo que o banco suporta, sem derrubar a loja;
- **Cache do catálogo:** vitrine e detalhes de produto, que mudam pouco, podem ser servidos por um cache como o Redis, reduzindo consultas ao banco;
- **Réplicas de leitura:** consultas de catálogo e busca podem ser enviadas a réplicas do PostgreSQL, deixando o banco principal para os pedidos;
- **Balanceador de carga e mais instâncias:** depois de resolvido o gargalo do banco, a aplicação pode ser escalada horizontalmente;
- **CDN para imagens e arquivos estáticos:** reduz o tráfego que chega aos servidores da loja;
- **Teste de carga antes de cada evento:** repetir o perfil PicoBlackFriday a cada mudança relevante, para conhecer o limite antes dos clientes.

9. Justificativa das Decisões Técnicas

- **Prometheus e Grafana:** ferramentas abertas, amplamente usadas no mercado e integradas nativamente ao Spring Boot por meio do Micrometer;
- **Monitoramento interno e externo:** as métricas da aplicação mostram o que acontece por dentro, e o Blackbox confirma se o cliente consegue acessar o site;

- **Percentil p95 em vez da média:** a média esconde os clientes que esperam muito; o p95 mostra o tempo que 95% dos clientes não ultrapassam;
- **Campo for nas regras de alerta:** evita notificações falsas causadas por picos de poucos segundos;
- **Telegram como canal de alerta:** notificação imediata no celular, gratuita e simples de configurar com um bot;
- **Token fora do Git:** o arquivo com o token do bot é ignorado pelo repositório, evitando expor credenciais;
- **Docker Compose:** todo o ambiente sobe com um comando, é versionado e pode ser reproduzido em qualquer máquina com Docker;
- **Dashboard provisionado como código:** o dashboard e a fonte de dados são criados automaticamente, sem configuração manual;
- **Locust com perfil de carga:** permite reproduzir o mesmo cenário de Black Friday antes e depois de cada otimização, garantindo uma comparação justa.

10. Conclusão

O projeto demonstrou, de ponta a ponta, como transformar uma plataforma sem visibilidade em um sistema observável: a aplicação expõe métricas, o Prometheus coleta e avalia regras, o Grafana apresenta a saúde da plataforma em tempo real e o Alertmanager avisa a equipe no Telegram assim que um limite é ultrapassado.

Os testes reproduziram o problema da Black Friday e permitiram medir o limite da plataforma, entre 220 e 420 usuários simultâneos. Mais importante, o monitoramento evitou uma conclusão errada: aumentar conexões e CPU não resolveu, enquanto a causa real, a busca sem índice, foi identificada pelos painéis e corrigida, levando a taxa de falhas no pico de 31,2% para 0%.

Como limitação, os testes foram realizados em laboratório, em um único servidor compartilhado com outros projetos, com gargalos recriados de forma controlada e a latência do gateway de pagamento simulada. Em produção, os mesmos princípios se aplicam, com tempos de alerta mais longos, mais instâncias da aplicação e as otimizações propostas na seção 8.3.

Referências

PROMETHEUS. Documentação oficial. Disponível em: https://prometheus.io/docs/. Acesso em: 2026.

GRAFANA LABS. Grafana documentation. Disponível em: https://grafana.com/docs/grafana/latest/. Acesso em: 2026.

PROMETHEUS. Alertmanager. Disponível em: https://prometheus.io/docs/alerting/latest/alertmanager/. Acesso em: 2026.

PROMETHEUS. Blackbox Exporter. Disponível em: https://github.com/prometheus/blackbox_exporter. Acesso em: 2026.

SPRING. Spring Boot Actuator: Production-ready Features. Disponível em: https://docs.spring.io/spring-boot/reference/actuator/. Acesso em: 2026.

MICROMETER. Documentação oficial. Disponível em: https://docs.micrometer.io/. Acesso em: 2026.

LOCUST. Documentação oficial. Disponível em: https://docs.locust.io/. Acesso em: 2026.

POSTGRESQL. pg_trgm. Disponível em: https://www.postgresql.org/docs/current/pgtrgm.html. Acesso em: 2026.

DOCKER. Documentação oficial. Disponível em: https://docs.docker.com/. Acesso em: 2026.

BEYER, B. et al. Site Reliability Engineering: How Google Runs Production Systems. Sebastopol: O'Reilly Media, 2016. Cap. 6: Monitoring Distributed Systems.

Repositório no GitHub: https://github.com/Dieggo775/efecaf-monitoramento